

An Improving Algorithm for the Node-to-Set Disjoint Paths Problem on Bi-Rotator Graphs

Yu-Sheng Huang¹, Sheng-Lung Peng^{1,*}, Ro-Yu Wu²

¹Department of Computer Science and Information Engineering
National Dong Hwa University, Hualien 974, Taiwan

²Department of Industrial Management
Lunghwa University of Science and Technology, Taoyuan 333, Taiwan
*slpeng@mail.ndhu.edu.tw

Abstract

Given a source and a node set, the node-to-set disjoint paths problem is to find disjoint paths from source to all the vertices of the node set. In this paper, we propose an algorithm for the node-to-set disjoint paths problem on bi-rotator graphs. Our result improves a previous result from $O(n^5)$ to $O(n^3)$, where n is the dimension of the input graph.

1 Introduction

In recent years, the research of parallel processing and distributed computing [3, 9] has become a very active research topic due to the advances of hardware in the fields of microchips and integration. For instance, the Jaguar supercomputer is built by the Cray Company which connecting 180,382 nodes precisely. The computer network can be seen as a graph by treating the computers (respectively, connections) as the nodes (respectively, edges) of the graph. Therefore, many classes of graphs, for example: hypercubes, n -pancakes, (n, k) -DeBruijn graphs and (n, k) -Kautz graphs, have been conducted enthusiastically research. These classes of graphs have lots of research topics, such as Hamiltonian cycle [2], vertex coloring [10], node-to-set disjoint paths [4], shortest path routing [2], and adaptive fault-tolerant routing [11]. Algorithms for those topics have been well developed. In this paper, we focus on the node-to-set disjoint paths problem. This problem is to find the disjoint paths from a source to vertices in a node set. Since rotator graphs are directed, we consider the bi-rotator graphs that are bi-directed (or undirected). In [6], they proposed an $O(n^5)$ algorithm for solving this problem

on bi-rotator graphs. In this paper, we improve a part of their algorithm and make the time complexity be down to $O(n^3)$.

The rest of this paper is organized as follows. In Section 2, we give the definition of bi-rotator graphs and related properties. The main improvement of the algorithm in [6] is presented in Section 3. Finally, we give the conclusion in Section 4.

2 Preliminary

A bi-rotator graph R_n is an undirected graph with $n!$ nodes and each node has the unique label which is a permutation of $\{1, 2, \dots, n\}$. If there is an edge between node $a = (a_1 a_2 \dots a_n)$ and node $b = (b_1 b_2 \dots b_n)$, then $b_1 = a_2, b_2 = a_3, \dots, b_{i-1} = a_i, b_i = a_1, b_{i+1} = a_{i+1}, \dots, b_n = a_n$ or $a_1 = b_2, a_2 = b_3, \dots, a_{i-1} = b_i, a_i = b_1, a_{i+1} = b_{i+1}, \dots, a_n = b_n$ for $2 \leq i \leq n$. R_n consists of n subgraphs, $R_{n-1}^1, R_{n-1}^2, \dots, R_{n-1}^n$. The rightmost number of the unique label of each node of R_{n-1} is the same. Figure 1 shows some examples of n -rotator graphs.

Suppose the source is $a = (a_1 a_2 \dots a_n)$, then we denote the neighbor nodes $(a_2 a_3 \dots a_n a_1)$ and $(a_n a_1 a_2 \dots a_{n-1})$ of the source as R and L respectively. Without loss of the generality, we assume that the source is in R_{n-1}^n . For example, there are six cycles in the R_4 in Figure 1. Assume (1234) is the source, then (2341) and (4123) are R and L respectively.

By the definition of bi-rotator graphs, There are at most $2n - 3$ destinations since the degree of each node is $2n - 3$. There are $(n - 1)!$ cycles in R_n and n nodes in each cycle. Clearly, each node in R_n belongs to exactly one cycle and each node in subgraph R_{n-1} belongs to different cycle. For example, node $a = (a_1 a_2 \dots a_n)$ in R_n , then $(a_2 a_3 \dots a_n a_1), (a_3 a_4 \dots a_n a_1 a_2), \dots,$

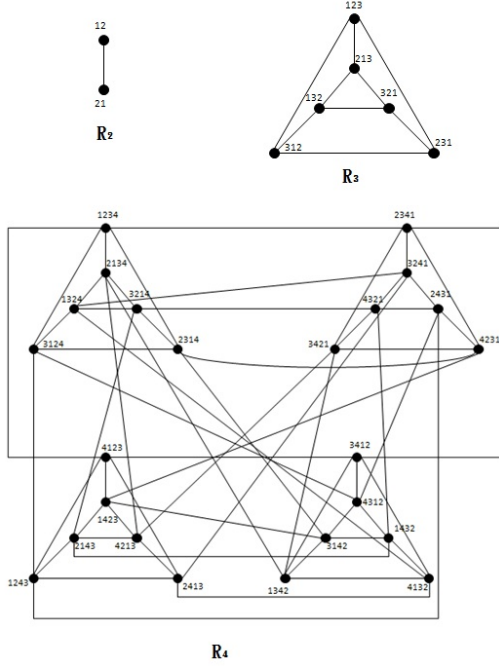


Figure 1: The examples of bi-rotator graphs, R_2 , R_3 , and R_4 .

$(a_n a_1 a_2 \cdots a_{n-1})$, $(a_1 a_2 \cdots a_n)$ are in the same cycle. The rightmost number of the unique label of each node is different. Thus, each node are in the different R_{n-1} . Let $N(a)$ denote the neighbors of a . Suppose nodes b and c are in different cycles in R_n . Since R_n consists of several R_3 's and each node in subgraph R_{n-1} belongs to different cycles, there exists $x, y \in N(b) \cup N(c)$ such that x and y are in different cycles. Let $C(a)$ denote the cycle in R_n containing a . For $n > 3$, if a is a destination and there is more than one destinations in $C(a)$, and there exists an $x \in N(a) \setminus C(a)$ which is not a destination, then there is at most one destination in $C(x)$. It is not hard to verify that the number of destinations in $N(a) \setminus C(a)$ is at most $2n - 5$. Moreover, $|N(a) \setminus C(a)| = 2n - 6$.

3 Algorithm

The idea of the algorithm proposed in [6] is a divide-and-conquer algorithm. It divides R_n into n R_{n-1} subgraphs and computes the disjoint paths in each R_{n-1} . It recursively divides a subgraph until it becomes an R_3 . Finally, the answer can be easily obtained. If there are remaining destinations that cannot connect to the source s in R_3 ,

then we connect the remaining destinations to R or L which are not connected to the destinations in the same subgraph in R_4 to R_n . The algorithm consists of two phases. In phase 1, there are four cases to classify the given bi-rotator graph into different solving methods. In phase 2, the remaining destinations which is not connecting to source are solved. Let $Queue_{RL}$ be a queue which records R and L which cannot connect to s for each round.

In the first phase of the algorithm, consider Figure 2 as an example. The destinations are colored gray. The leftmost cycle is the subgraph R_{n-1}^n of R_n . Since there is only one destination in the middle cycle, it can use the cycle itself to connect to R_{n-1}^n . However, there are more than one destinations in the rightmost cycle. Thus, only one destination can use this cycle to connect to R_{n-1}^n . Therefore, only four cases should be considered. Due to the limitation of pages, we omit the detail for the first three cases. For the detail, please refer [6].

To find the relation between destinations in the same cycle, we use three circles with destinations shown in Figure 2 to help us to understand. At the beginning, we pick an arbitrary destination d , and then use $C(d)$ to connect to A in R_{n-1}^n . Then we traverse $C(A)$ from A by a BFS and count the shortest distance from A to each vertices in $C(A)$. Then we partition the destinations into the three circles. The destinations in $C(A)$ whose distances to A are zero will be in the leftmost circle. And the destinations in $C(A)$ whose distances to A are greater than zero and are smallest will be in the middle circle. The remaining destinations in $C(A)$ are in the rightmost circle.

Before starting to illustrate the algorithm, we first define some notation used in Case 4. A_0 is the number of destinations in the leftmost circle, and A_1 is the number of destinations in the middle circle except the destinations in $C(s)$. $Queue_1$ is a queue that stores the destinations in the middle circle except the destinations in the $C(s)$, and $Queue_2$ is a queue that stores the destinations in the rightmost circle except the destinations in the $C(s)$. N is the number of the destinations in $R_n - R_{n-1}^n$ which have linked to R_{n-1}^n . $Queue_s$ is a queue that store the destinations in $C(s)$, and $Queue_{NRL}$ is a queue that stores R and L in this round. Moreover, $Queue_{NCD}$ is a queue that stores the used circles of the neighbor which is a destination. OP is a set storing the routes in this round. And UC is a set stores the corresponding cycles of the destinations in $R_n - R_{n-1}^n$ linked to R_{n-1}^n .

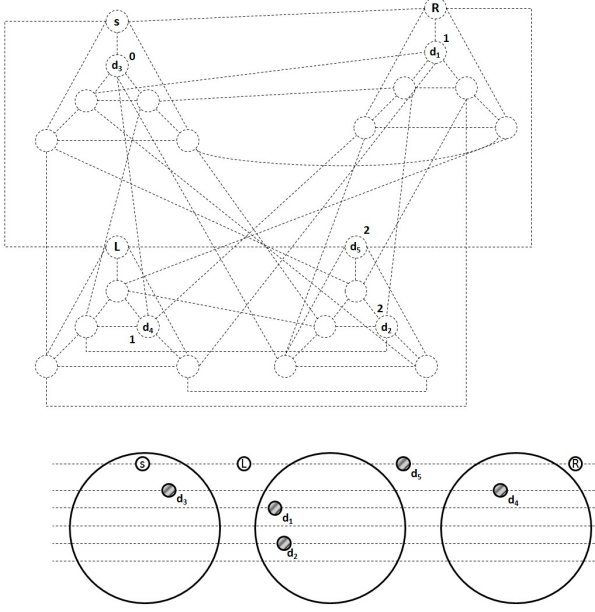


Figure 2: Find the relation between destinations.

Case 1: All destination in the R_{n-1}^n .

Case 2: Only one destination, d_a , is in $R_n - R_{n-1}^n$. We divide this case into two subcases.

1. Destination and R in the same subgraph.
2. Destination and R in different subgraphs.

Case 3: Two destinations, d_a and d_b , are in $R_n - R_{n-1}^n$. We divide this case into two subcases.

1. The destinations are in different subgraphs.
2. The destinations are in the same subgraph. There are two subcases.
 - (a) The destination and R are in different subgraphs.
 - (b) The destination and R are in the same subgraph R_{n-1}^b .

Case 4: There are more than two destinations in $R_n - R_{n-1}^n$. We have two subcases.

1. There are less than $2n - 5$ destinations in R_{n-1}^n . We divide this case into five parts.

Part 1: Connect the destinations in $R_n - R_{n-1}^n$ to R_{n-1}^n until there is no more destination to connect to R_{n-1}^n or the number of the destinations connect to R_{n-1}^n is greater

than or equal to $2n - 3$. There are 6 substeps.

- (a) If $Queue_S \neq \phi$ and $Queue_{NRL} \neq \phi$, then we do this step until $Queue_S = \phi$ or $Queue_{NRL} = \phi$. If not, then $Queue_2 = Queue_2 \cup Queue_S$. See Figure 3.

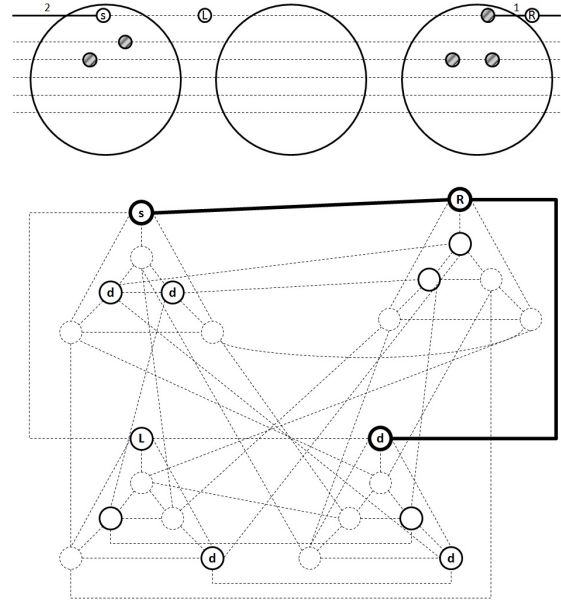


Figure 3: Substep (a)

- Connect d to the closest one of L and R by using $C(s)$. We assume d connected to R .
 - Connect R to s by using $C(s)$.
 - Output the route from d to s computed in Step 1 and 2.
 - Pop d from $Queue_S$.
 - Pop R which connect to s from $Queue_{NRL}$.
 - $N = N + 1$.
 - If $N = 2n - 3$ then jump to Part 2.
- (b) If $Queue_1 \neq \phi$, then we do this step until $|Queue_1| = \phi$. See Figure 4.
 - Pop d from $Queue_1$ and use $C(d)$ to connect to A in R_{n-1}^n .
 - Output the route from d to A computed in Step 1 and 2 to OP .

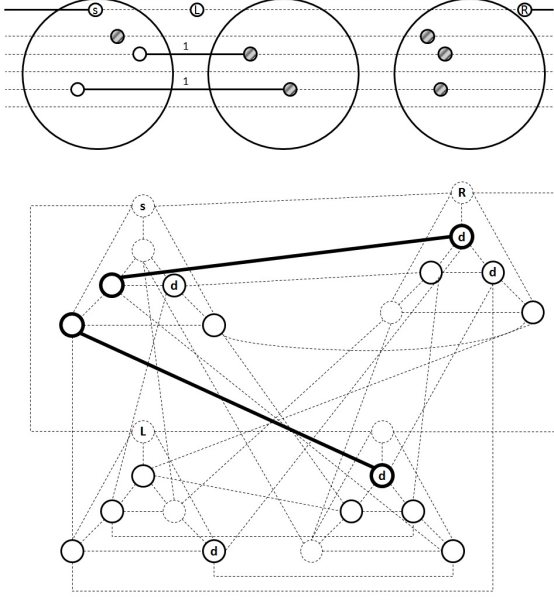


Figure 4: Substep (b)

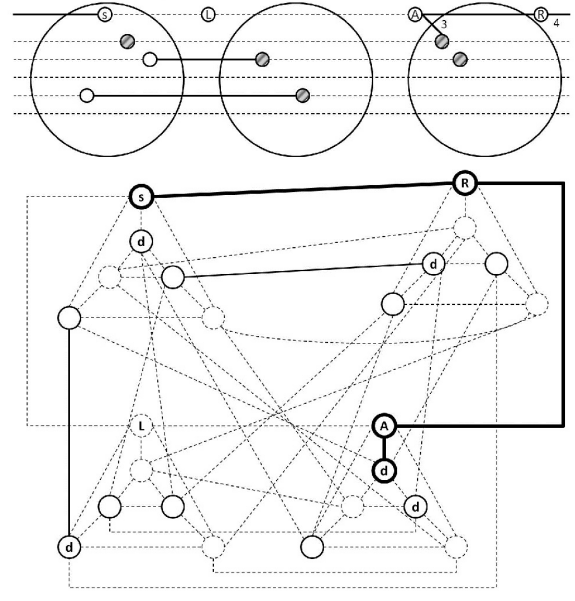


Figure 5: Substep (c)

- $N = N + 1$.
- If $N = 2n - 3$, then jump to Part 2.
- (c) If $Queue_{NRL} \neq \phi$, then we do this step for $|Queue_2|$ rounds. If not, we go to the next step. See Figure 5.
 - Pop d from $Queue_2$.
 - If there exists a neighbor node $A \in C(s)$ of d and A has not been marked, then do the following 6 steps. If not, jump to the last step.
 - To connect d to a , we connect d to closest one of L and R by using $C(s)$. We assume d connects to R .
 - R connects to s by using $C(s)$.
 - Pop d which connects to R or L in $Queue_{NRL}$.
 - Output the route from s to d .
 - $N = N + 1$.
 - If $N = 2n - 3$, then jump to Part 2.
 - Push d into $Queue_2$.
- (d) If $Queue_2 \neq \phi$, then we do this step until $Queue_2 = \phi$. See Figure 6.
 - Pop d from $Queue_2$.

- If there exists a neighbor A of d which is not marked used, then we do the following 7 steps. If not, then we jump to the last step.
- Connect d to A .
- Connect A to $B \in R_{n-1}^n$ by using $C(A)$.
- Mark all the vertices in $C(A)$ which have been used.
- Output a route of the destination d connected to node B to OP .
- Push $C(A)$ into UC .
- $N = N + 1$.
- If $N = 2n - 3$ then go to Part 2.
- Push d into $Queue_{NCD}$.
- (e) If $Queue_{NCD} = \phi$ then jump to Part 2. See Figure 7.
 - Pop d from $Queue_{NCD}$.
 - If d does not have a neighbor C which is not a destination, then push d into $Queue_{NCD}$ and jump to Substep (f).
 - P_1 is the previous route in OP . $P_2 = C(C) \setminus P_1$.
 - If $C \in P_1$, then $P_1 = P_2$.
 - d connects to C .

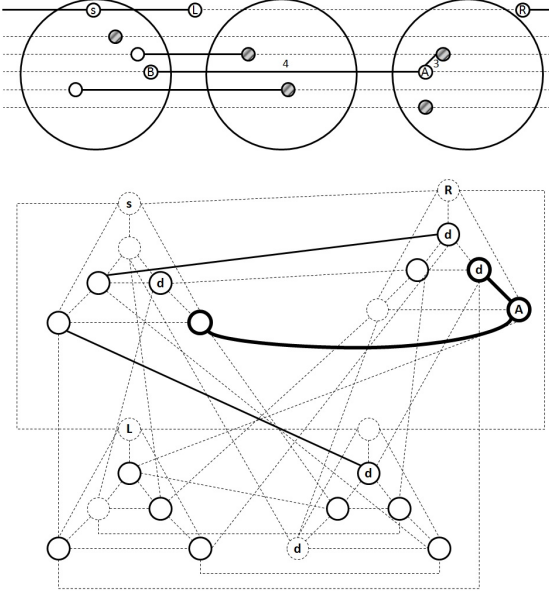


Figure 6: Substep (d)

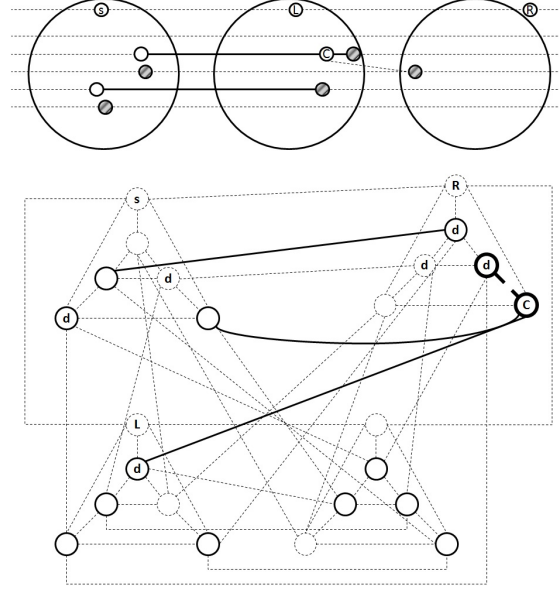


Figure 7: Substep (e)

- If node C has a neighbor $B \in C(s)$ which is not connected to R_{n-1}^n and $Queue_{NRL} \neq \phi$, then we do Steps 3 to 6 of Substep (c) by replacing d to C in this substep and A to B in this substep. Then jump to Part 2.
 - If node C have a neighbor B where $C(B)$ is not used, then we do Steps 3 to 7 of Substep (d) by replacing d to C in this substep and A to B in this substep. Then jump to Part 2.
- (f) If $Queue_{NCD} \neq \phi$, then do the following; otherwise, go to Part 2.
- Pop d in $Queue_{NCD}$.
 - d connects to a neighbor A which is not a destination by using $C(d)$.
 - If node A have a neighbor $B \in C(s)$ which is not connected to R_{n-1}^n , then do Step 3 to 7 of Substep (c) where d is replaced by A and A in Substep (c) is replaced by B . Then jump to Part 2.
 - Do Steps 2 to 7 of Substep (d) where d is replaced by A . Then jump to Part 2.

- Do Steps 2 to 7 of Substep (e) where d is replaced by A . Then jump to Part 2.
- Part 2:** If $L \in Queue_{NRL}$, then s uses $C(s)$ to connect to L and then L connects to $A \in C(A)$ in UC .
- s connects to L and then L connects to A in $C(A)$ in UC .
 - $B \in C(A)$ is the node nearest to d . P_1 is a sub path from d to B in OP . P_2 is the path from d to R_{n-1}^n through B in OP . P_3 is a path from A to B .
 - $OP = OP \setminus P_2$.
 - Output the $P_1 + P_3$.
- Part 3:** If $R \in Queue_{NRL}$, then s uses $C(s)$ to connect to R and then R connects to A in $C(A)$ in UC .
- s connects to R .
 - R connected to $A \in C(A)$ in UC .
 - $B \in C(A)$ is the node nearest to d .
 - If d is connected to L , then jump to Step 2.
 - P_1 is a sub path from d to B in OP . P_2 is the path from d to R_{n-1}^n through B in OP . P_3 is a path from A to B .

- $OP = OP \setminus P_2$.
- Output $P_1 + P_3$.

Part 4: If the number of the destinations in R_{n-1}^n plus the number of the corresponding destinations from $R_n - R_{n-1}^n$ to R_{n-1}^n is greater than $2n - 5$, then delete the paths in OP until equal to $2n - 5$.

Part 5: Output all the paths in OP and keep the sources in R_{n-1}^n as the destinations in next round.

Case 4.2: There are more than $2n - 5$ destinations in R_{n-1}^n . We omit the detail for this version due to the space limitation.

After proposing the solving steps for Phase 1, now we are going to show the solving steps for Phase 2.

Step 1. If there are still destinations in $R_n - R_{n-1}^n$ that are not connected to R_{n-1}^n , then connect the destinations to previous recorded L or R pop from $Queue_{RL}$. Finally, output the path which starts from the destination through R or L to s .

Step 2. If $n = 3$, then we use brute force method to find the path.

Step 3. If there are still some destinations in R_3 not linking to s , then do Step 1.

Step 4. If $n \neq 3$, then $n = n - 1$.

The correctness of the algorithm can be obtained easily. We now analyze the time complexity of our algorithm. At the beginning, we first find each destination distributed in which subgraph by checking the rightmost number of the label of each destination. Since there are $2n - 3$ nodes, the time complexity is $O(n)$. By the analysis in [6], Case 1 costs $O(1)$ time; Case 2 costs $O(n^2)$ time; and Case 3 costs $O(n^2)$ time. In Case 4, we first find which cycles the destinations are located. The worst case is that there are $2n - 3$ destinations in different cycles and each destination uses the cycle itself to the source. Thus, this step costs $O(n^2)$ time. There are five parts in Case 4.1. In Part 1, we have six substeps. To connect destination in the $C(s)$ to s costs $O(n)$ time and $O(1)$ time for the remaining operations. Since there are at most n destinations in $C(s)$, the time complexity of Step 1 is $O(n^2)$. Similar to Substep 1, Substep 2 costs $O(n^2)$ because there are at most $O(n)$ destinations in $Queue_1$ and each destination

needs to pass through all subgraphs to connect to R_{n-1}^n by using its own cycle which costs $O(n)$ time. In $Queue_2$, all destinations need to check all their neighbors to see if they can connect to the source by using their neighbors. Therefore, the time complexity of Substep 3 is $O(n^2)$. Since there are at most $O(n)$ destinations in $Queue_2$ to check at most $O(n)$ neighbors and use $O(1)$ time to connect to the neighbor which is not used and using this cycle to connect to R_{n-1}^n costs $O(n)$ time, the total time is $O(n) * (O(n) + O(1) + O(n)) = O(n^2)$ for Substep 4. For Substep 5, there is only one neighbor of the destination. Hence, it needs $O(n)$ time to check and $O(n)$ time to connect to R_{n-1}^n by using the methods of Steps 3 and 4. Therefore, the total time complexity is $O(n)$. In Substep 6, we use $O(1)$ time for finding the neighbor which is not a destination and connect to R_{n-1}^n by using the methods of Steps 3, 4, and 5. Thus, Step 6 costs $O(n)$.

For Part 2, it is to link L to the destination. Step 1 is to link each two nodes in the graph. So, it needs $O(n^2)$ time. The remaining steps need $O(n)$ time. Therefore, the total time complexity of Part 2 is $O(n^2)$. The time complexity of part 3 is the same as part 2. Part 4 needs to delete at most 2 paths of destination linking to R_{n-1}^n , this costs $O(n)$. Part 5 is to output the path in the OP which costs $O(n)$. Case 4.2 is similar to Case 3. Thus, the time complexity of Case 4.2 is $O(n^2)$. The time complexity of Phase 2 is $O(n^2)$. Therefore, the algorithm total time complexity is $O(n^3)$ because all the steps takes time at most $O(n^2)$ and we need to execute n rounds. Hence, the algorithm takes $O(n^3)$ time.

4 Conclusion

In this paper, we propose an algorithm for solving the node-to-set disjoint paths problem on birotator graphs. Our algorithm improves the algorithm proposed in [6] from $O(n^5)$ to $O(n^3)$. The main improvement is the Case 4. We improve it from $O(n^4)$ to $O(n^2)$.

References

- [1] A. A. Bertossi, Dominating Sets for Split and Bipartite Graphs, *Information Processing Letters* 19, pp. 37-40, 1984.
- [2] P.F. Corbett, Rotator graphs: An efficient topology for point-to-point multiprocessor net-

- works, *IEEE Trans. Parallel and Distributed Systems*, 3, pp. 622-626, 1992.
- [3] Q.-P. Gu and S. Peng, Node-to-set disjoint paths problem in star graphs, *Information Processing Letters* 62, pp. 201-207, 1997.
 - [4] Y. Hamada, F. Bao, A. Mei, and Y. Igarashi, Nonadaptive fault-tolerant file transmission in rotator graphs, *IEICE TRANS. Fundamentals of Electronics* E79-A, pp. 477-482, 1996.
 - [5] K. Kaneko, An Algorithm for Node-to-Set Disjoint Paths Problem in Burnt Pancake Graphs, *IEICE TRANS. INF. & SYST.* E86-D, pp. 2588-2594, 2003.
 - [6] K. Kaneko, An Algorithm for Node-to-Set Disjoint Paths Problem in Bi-Rotator Graphs, *IEICE TRANS. INF. & SYST.* E89-D, pp. 647-653, 2006.
 - [7] K. Kaneko and Y. Suzuki, An Algorithm for Node-to-Set Disjoint Paths Problem in Rotator Graphs, *IEICE TRANS. INF. & SYST.* E84-D, pp. 1155-1163, 2001.
 - [8] K. Kaneko and Y. Suzuki, Node-to-Set Disjoint Paths Problem in Pancake Graphs," *IEICE TRANS. INF. & SYST.* E86-D, pp. 1628-1633, 2003.
 - [9] M.O. Rabin, Efficient dispersal of information for security, load balancing, and fault tolerance, *JACM* 36, pp. 335-348, 1989.
 - [10] P.J. Wan, Conflict-free channel set assignment for an optical cluster interconnection network based on rotator digraphs, *Theoretical Computer Science* 207, pp. 193-201, 1998.
 - [11] P.M. Yamakawa, H. Ebara, and H. Nakano, A routing algorithm in faulty n -rotator graph and its performance evaluation, *Trans. IPSJ* 56, pp. 1511-1519, 1995.